

C04 -AT3 -DEBUGGING

Q1 Inheritance and Constructor Order

10 marks

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    Vehicle() {
        cout << "Vehicle Constructor" << endl;
    }
};

class Car : Vehicle {
public:
    Car() {
        cout << "Car Constructor" << endl;
    }
};

int main() {
    Car c;
    return 0;
}
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class Vehicle {
4 public:
5     Vehicle() {
6         cout << "Vehicle Constructor" << endl;
7     }
8 };
9 class Car : public Vehicle {
10 public:
11     Car() {
12         cout << "Car Constructor" << endl;
13     }
14 };
15 int main() {
16     Car c;
17     return 0;
18 }
```

Input

Vehicle
Car

Output



Q2 Encapsulation and Setter Validation

10 marks

```
#include <iostream>
using namespace std;

class BankAccount {
public:
    double balance;

    void deposit(double amount) {
        balance += amount;
    }

    void withdraw(double amount) {
        balance -= amount;
    }

    void display() {
        cout << balance << endl;
    }
};

int main() {
    BankAccount account;
    double initial, deposit, withdraw;
    cin >> initial >> deposit >> withdraw;
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class BankAccount {
4 public:
5     double balance;
6     void deposit(double amount) {
7         balance += amount;
8     }
9     void withdraw(double amount) {
10        balance -= amount;
11    }
12    void display() {
13        cout << balance << endl;
14    }
15 };
16 int main() {
17     BankAccount account;
18     double initial, deposit, withdraw;
19     cin >> initial >> deposit >> withdraw;
20     account.balance = initial;
21     account.deposit(deposit);
22     account.withdraw(withdraw);
23     account.display();
24     return 0;
25 }
```

Input

1000 500 200

Output

Q3 Virtual Destructor and Memory Leak

10 marks

```
#include <iostream>
using namespace std;

class Parent {
public:
    ~Parent() {
        cout << "Parent Destructor" << endl;
    }
};

class Child : public Parent {
public:
    ~Child() {
        cout << "Child Destructor" << endl;
    }
};

int main() {
    Parent *p = new Child();
    delete p;
    return 0;
}
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class Parent {
4 public:
5     virtual ~Parent() {
6         cout << "Parent Destructor" << endl;
7     }
8 };
9 class Child : public Parent {
10 public:
11     ~Child() {
12         cout << "Child Destructor" << endl;
13     }
14 };
15 int main() {
16     Parent *p = new Child();
17     delete p;
18     return 0;
19 }
```

Input

Child
Parent

Output

Q4 Abstract Class and Runtime Polymorphism

10 marks

```
#include <iostream>
using namespace std;

class Employee {
public:
    void calculateSalary() {
        cout << "Employee Salary" << endl;
    }
};

class FullTimeEmployee : public Employee {
    double salary;
public:
    FullTimeEmployee(double s) {
        salary = s;
    }

    void calculateSalary() {
        cout << salary << endl;
    }
};

class PartTimeEmployee : public Employee {
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class Employee {
4 public:
5     virtual void calculateSalary() {
6         cout << "Employee Salary" << endl;
7     }
8     virtual ~Employee() {}
9 };
10 class FullTimeEmployee : public Employee {
11     double salary;
12
13 public:
14     FullTimeEmployee(double s) {
15         salary = s;
16     }
17 void calculateSalary() override {
18     cout << salary << endl;
19 }
20 };
21 class PartTimeEmployee : public Employee {
22     int hours;
23     double rate;
24
25 public:
26     PartTimeEmployee(int h, double r) {
```

Input

```
50000
20 500
```

Output

```
#include <iostream>
using namespace std;

class Shape {
public:
    void area() {
        cout << "Shape Area" << endl;
    }
};

class Rectangle : public Shape {
public:
    void area() {
        int l, b;
        cin >> l >> b;
        cout << l * b << endl;
    }
};

class Circle : public Shape {
public:
    void area() {
        int r;
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class Shape {
4 public:
5     virtual void area() {
6         cout << "Shape Area" << endl;
7     }
8 };
9 class Rectangle : public Shape {
10 public:
11     void area() override {
12         int l, b;
13         cin >> l >> b;
14         cout << l * b << endl;
15     }
16 };
17 class Circle : public Shape {
18 public:
19     void area() override {
20         int r;
21         cin >> r;
22         cout << 3.14 * r * r << endl;
23     }
24 };
25 int main() {
26     Shape *s;
27     Rectangle r;
```

Input

```
5 4
3
```

Output

